# GCP SLA compliance monitor: design and prototype

I built a working tool called `slawatch`, deployed a real Cloud Run service to test it against, and ran it end to end. The project is `slawatch-live-demo`, the service is `slawatch-demo` in `us-central1`, and the actual run output is sitting in `reports/`. The code is in the same repo. This doc walks through how I thought about it.

I went a bit past the suggested hour, about two hours total. I wanted something that actually queries Cloud Monitoring and produces a real verdict, not a description of a tool that could exist.

The order I worked in is roughly the order of this doc:

1. How I framed the problem
2. What "SLA compliance" actually means (this took the longest)
3. Why I picked Cloud Run
4. Architecture
5. Implementation decisions
6. How a team uses this day to day
7. Things I left out
8. Risks and limits

## How I framed it

I read the brief twice before writing any code. Two phrases stuck:

> *"What you define as SLA compliance for your chosen service(s) and why."*
>
> *"How you'd make it usable for a team sharing reports or configs."*

The first one is the actual question. SLA compliance sounds settled. You'd think it just means "is the service up?" But the moment I sat down to write the code, I realized it isn't. The second one is what separates a one-off script from a tool a team can actually use. I took both seriously.

## What "SLA compliance" actually means

This is the part I want to land first, because everything else follows from it.

GCP publishes a contractual SLA for each service. I read the current Cloud Run one (`https://cloud.google.com/run/sla`) carefully before writing any code. Here's what it actually says:

- Standard regions get 99.95% Monthly Uptime Percentage. Mexico (`northamerica-south1`) and Stockholm (`europe-north2`) are 99.9%. GPU services get 99.95% with zonal redundancy and 99.5% without.
- Downtime is when error rate goes above 1% for one or more consecutive minutes, and there have to be at least 100 valid requests in that minute for it to count.
- Monthly uptime is `(total minutes - downtime minutes) / total minutes`.
- Below the floor, Google issues credits at three tiers: 10% for uptime in [99.0%, 99.95%), 25% for [95%, 99%), and 50% for anything below 95%.

- The customer has 30 days to claim, and they have to supply log files.
- The SLA explicitly excludes errors caused by the customer's own code, third-party software, quota limits, and abuse.

That last bullet is what shifted how I was thinking about this. The SLA isn't measuring "how broken was the service." It's measuring "how broken was it in ways Google takes responsibility for." If your code throws because you blew through a quota, that's on you. If a client calls a deprecated endpoint and gets a 410, that's on you. The SLA is specifically about Cloud Run infrastructure failures, which is why the contract says 5xx responses "due to a failure of the Cloud Run infrastructure", not just any 5xx.

In practice I went with `run.googleapis.com/request_count` filtered on `response_code_class = "5xx"` as the closest tractable proxy. I could filter further to specific codes, but I don't think it's worth the complexity. Most actual Cloud Run infra failures land in 5xx, and the SLA's own measurement framework uses the same bucket.

So my working definition for the tool came out as:

> *Monthly Uptime Percentage over a configurable rolling window, computed minute-by-minute per the contract. A minute is Downtime when at least 100 valid requests arrive AND more than 1% return a 5xx. The result gets compared against two thresholds: the published SLA floor (a hard line; below it, Google owes a credit) and the team's own SLO (a stricter internal target, used to catch problems before users feel them).*

That two-threshold thing matters to me. The SLA is written for Google's protection, not for the team's reliability practice. If a team only watches the SLA floor, they only react to actual contract breaches, by which point users have already had a bad week. The SLO is the warning track.

A couple of things I deliberately left out of the first version:

I didn't build multi-window burn-rate alerts, the SRE workbook approach with paired short and long windows. That's the right model for paging, but it's a different tool from a compliance report. A compliance report answers "are we within SLA for the billing month?" Burn-rate alerts answer "are we burning the error budget too fast right now?" They belong in Cloud Monitoring alert policies, not in a CLI that runs nightly. Mixing them would muddy both.

I also didn't treat 4xx as a compliance signal. Some teams want 4xx visibility for quality reasons, which is reasonable, but 4xx is excluded from the SLA (those are client errors), so rolling them into the compliance number would conflate two different things. I left it as a future config flag.

## Why Cloud Run

The brief listed Compute Engine, Cloud Storage, BigQuery, and Cloud Run. I picked Cloud Run for a few reasons.

The SLA is short and self-contained. I could read it in one sitting and have a clear mental model. Compare that to Compute Engine, where the SLA depends on multi-zone redundancy patterns, instance group configurations, and a bunch of architectural assumptions about how things are deployed. Modeling that correctly would have been a much bigger project.

The metric exists in exactly the right shape. `run.googleapis.com/request_count` with

`response_code_class` is literally the SLA's measurement primitive. No bridging, no log-based metrics, no proxy SLIs. I just had to query it.

It's also what teams are deploying today. Building something new on GCP right now, I'd default to Cloud Run before I'd reach for managed instance groups.

Cloud Storage was tempting because its SLA has the same structure: uptime percentage, credit tiers, 30-day claim window. So adding it later means one new fetcher class. I designed for that.

## Architecture

The flow is small and one-directional:

```
                +-----------------------+
                |   config.yaml         |
                |   (in git)            |
                +----------+------------+
                           |
                           v
                +-----------------------+
                |   Config loader       |
                |   (pydantic)          |
                +----------+------------+
                           |
                           v
      +--------------------+--------------------+
      |          For each target                |
      |   +--------------------------------+    |
      |   |  CloudRunFetcher               |    |
      |   |  Cloud Monitoring API          |    |
      |   |  retries + error handling      |    |
      |   +----------------+---------------+    |
      |                    |                     |
      |                    v                     |
      |   +--------------------------------+    |
      |   |  Evaluator                     |    |
      |   |  per-minute downtime detection |    |
      |   |  uptime, error budget, verdict |    |
      |   +----------------+---------------+    |
      +--------------------+--------------------+
                           |
                           v
                +-----------------------+
                |  Reporters            |
                |  markdown / json      |
                +----------+------------+
                           |
                  +--------+--------+
                  |                 |
                  v                 v
             reports/          CI artefacts
             in repo           (JSON)
```

Three things I want to call out about why this looks the way it does.

The config layer means the same code runs locally, in Docker, and in CI without any flag changes. Authentication uses Application Default Credentials, which means no service account JSON files in

CI when paired with Workload Identity Federation. I did the `gcloud auth application-default login` flow on my laptop, mounted the credential file into Docker, and the same code ran in both places.

The fetcher is an abstract base class. I knew adding more services (Cloud Storage especially) was likely, so I drew the line such that `MetricFetcher.fetch(...)` returns minute buckets and a latency series, and the evaluator doesn't care where they came from. Adding Cloud Storage is one new file.

The evaluator is pure. Raw minute buckets in, verdict out, no I/O. That's what makes it cheap to test against the SLA wording precisely. The unit tests cover the things the SLA actually carves out: minutes that don't have enough traffic to count, consecutive downtime grouping into one period, the region-specific floors for Mexico and Stockholm.

## Implementation notes

A handful of decisions worth surfacing.

### The Cloud Monitoring query

The query I ended up with:

```
filter:        metric.type = "run.googleapis.com/request_count"
               AND resource.type = "cloud_run_revision"
               AND resource.labels.service_name = "<name>"
               AND resource.labels.location = "<region>"

aggregation:   alignment_period      = 60s
               per_series_aligner    = ALIGN_DELTA
               cross_series_reducer  = REDUCE_SUM
               group_by_fields       = [
                   resource.labels.service_name,
                   metric.labels.response_code_class,
               ]
```

Two things to watch for here.

The metric is cumulative, so `ALIGN_DELTA` is the right aligner for per-minute counts. `ALIGN_RATE` would give a per-second rate, which I'd then have to multiply by 60 to recover counts. Pointless. The other trap is grouping by `revision_name`, which gives one series per revision. That's fine for traffic charts but wrong for SLA computation. The SLA is at the service level, not the revision level, so I sum across revisions.

I considered pushing the ratio computation into the API itself with a denominator filter. Cloud Monitoring supports it. But the response shape gets harder to reason about, and the per-minute downtime detection logic is much clearer when I have the raw counts in Python. I'd rather do the math in code I can read than in a Monitoring filter expression I have to reverse-engineer.

### Latency

I used `ALIGN_PERCENTILE_99` against `run.googleapis.com/request_latencies`, with the alignment period set to the entire evaluation window. That gives one p99 number for the whole window, which is what the report needs.

A latency breach demotes a service from PASSING to WARN, but never to BREACH. The Cloud Run SLA is availability-only. Latency is the team's SLO, not Google's commitment. I think keeping that separation visible in the verdict is more useful than blending them. When someone reads the report and sees WARN, they immediately know whether it's a vendor problem (SLA-floor side) or their own SLO that needs attention.

### Retries

The fetcher retries `ServiceUnavailable`, `DeadlineExceeded`, and `InternalServerError` with exponential backoff (1s initial, doubling, 3 attempts max). `PermissionDenied` and `InvalidArgument` surface immediately because retrying isn't going to fix anything. I kept this small on purpose: enough to handle a transient Cloud Monitoring hiccup, not enough to hide a real auth, quota, or filter problem.

### Eligible minutes

The SLA only counts a minute toward Error Rate if it has at least 100 valid requests. The tool tracks "eligible minutes" separately from "total minutes" and includes the count in every report. This matters more than it sounds. A low-traffic service running at night will have a lot of minutes that don't count, and someone looking at the report needs to be able to see that ratio rather than just trust the uptime percentage on its own.

### Output

The tool writes markdown and JSON. Markdown is for humans reading the report in GitHub or as a CI artifact. JSON is for downstream pipelines: BigQuery ingestion, dashboards, anything that wants structured data. I had a third format in there at one point and pulled it, because the markdown and JSON cover the actual use cases I could think of, and adding more formats just for theoretical extensibility felt like overengineering.

The `reports/` directory has actual output from a live run against `slawatch-demo`: verdict passing, uptime 100%, p99 latency 10ms. Real data, real metrics, no mocks anywhere in the path.

## Team usability

The config sits in git. Every target, threshold, and SLO is reviewable in a pull request. Nobody owns "the secret YAML on someone's laptop," and the `validate` subcommand can run as a pre-commit check to catch schema errors before they hit CI.

Reports are simple files. The included GitHub Actions workflow runs daily, validates the config, and uploads the report folder as a CI artifact. If the GCP Workload Identity variables are set, it also runs the live Cloud Monitoring check. Different audiences get the format they need: humans read markdown and downstream pipelines pick up JSON.

Exit codes are stable: 0 clean, 1 warnings, 2 SLA breach, 3 runtime error. A breach shows up as a red CI job in whatever notification system the team already uses. The tool doesn't need to know what that system is, and I think that's the right boundary. A red job can feed chat, paging, or GitHub notifications without slawatch needing a separate integration for each one.

CI uses Workload Identity Federation, so there are no service account keys to rotate. The service account needs `roles/monitoring.viewer` and nothing else.

## What I left out

Multi-window burn-rate alerts. I covered why above. They belong in Cloud Monitoring alert policies, not in a CLI report. Real-time alerting and rolling compliance reporting are different jobs.

Historical storage. Right now each run writes a new file, and GitHub Actions keeps uploaded artifacts for a retention window. That's enough for reviewing a recent run, but not great for trend dashboards. The natural extension is a JSON-emitting reporter that writes to BigQuery, which is one more class without changing anything else.

Custom 4xx tracking. The SLA excludes them, so I excluded them, but a `count_4xx` flag on each target would be a reasonable addition for teams that want quality signals.

Cloud Storage and Compute Engine fetchers. Architecturally cheap to add. I didn't write them because the brief said one service is enough, and Cloud Run was the most interesting one to model.

A Prometheus exporter. I thought about this and decided no. A Prometheus exporter means a service to host, retention to manage, and alert policies to tune. The brief specifically warned against overkill stacks, and that would be exactly that.

## Risks and limits

Three things I'd want anyone reviewing this to push on.

The SLA catalog is hardcoded. The `sla_catalog.py` module embeds the published Cloud Run values, with a `VERIFIED_AT` date that ships in every report. Google revises SLAs without much warning, so operationally someone needs to re-verify the catalog whenever there's a noteworthy GCP launch. I considered scraping the SLA HTML pages, but they aren't structured for it, and I'd rather have a manual periodic check than fragile parsing that breaks silently.

The 100-request floor is a real limitation for off-hours services. A service that gets 50 requests an hour at night will have most minutes flagged as ineligible, and the verdict can be misleading because "no traffic" and "perfect uptime" look identical to this kind of measurement. The tool surfaces the eligible minute count, but anyone reading the report needs to know what that means. The right mitigation is pairing slawatch with a Cloud Monitoring uptime check. The tool's job is contractual compliance, not synthetic liveness.

The minute-level computation depends on Cloud Monitoring returning aligned minute buckets. In practice it does, but if the underlying time series has gaps (which happens with very low traffic), the gaps get silently treated as "not down." That's correct under the SLA, but it means the tool can't tell you "we got zero requests for an hour because the ingress was broken." An uptime check covers that case. I'd add one as a paired feature, with the result folded into the same compliance verdict.

## What I'd do with another week

In rough priority order:

1. Cloud Storage fetcher. Same SLA shape, clean extension, useful coverage.
2. BigQuery destination for the JSON reports, plus a Looker Studio template. That gives the team trend lines without a custom UI.

3. A Cloud Monitoring alert policy generator. Given the slawatch config, emit Terraform that creates burn-rate alerts at 1h-fast and 6h-slow windows, so the alert layer and the report layer share the same source of truth.
4. A "compare to previous report" view for month-over-month changes. Useful for retrospectives.

Code is in the repo, tests pass (`pytest -v` is 25 passing), and the `reports/` folder has live output from the real GCP project. Happy to walk through any of this on a call.

## References


- Cloud Run SLA: `https://cloud.google.com/run/sla`
- Cloud Run monitoring overview: `https://cloud.google.com/run/docs/monitoring`
- Cloud Monitoring filter syntax: `https://cloud.google.com/monitoring/api/v3/filters`
- ListTimeSeries reference: `https://cloud.google.com/monitoring/api/ref_v3/rest/v3/projects.timeSeries/list`
- Request-response SLI metrics: `https://cloud.google.com/stackdriver/docs/solutions/slo-monitoring/sli-metrics/req-resp-metrics`
- Workload Identity Federation for GitHub Actions: `https://github.com/google-github-actions/auth`
- Google SRE Workbook, Implementing SLOs: `https://sre.google/workbook/implementing-slos/`